Synthetizing textual information with embeddings

Lino Galiana
Version of October 16, 2026

To be able to use textual data in machine learning algorithms, we need to vectorize text, i.e. transform that object into numerical data. In this tutorial, we will compare different vectorization methods, using a prediction task: *can we predict a literary author from extracts of his texts?* Among these methods, we will explore the `Word2Vec` model, which allows us to exploit the latent structures of a text by building word embeddings.

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 Introduction **1**
1.1 Data Used 2
1.2 Required Packages 2
2 Data Preparation **3**
2.1 Preprocessing 3
2.2 Encoding the Target Variable 5
2.3 Creating the Training and Test Sets 5
3 Vectorization Using the *Bag of Words* Approach **5**
3.1 Finding the Most Similar Text 22
3.2 Finding the Closest Author: An Introduction to the Naive Bayes Classifier 25
4 The Word2Vec model, a more synthetic representation **38**
4.1 Towards a more synthetic representation of language 38
4.2 Semantic relationships between terms 39
4.3 How are these models trained? 41
4.4 Related models 41
5 *Transformers*: a richer representation of language **42**
Informations additionnelles 44
Bibliography **46**

⚠️ Warning

Ce chapitre va évoluer prochainement.

1 Introduction

This page builds on certain aspects presented in the introductory section. We will advance our understanding of NLP issues through language modeling.

We start from the conclusion noted at the end of the previous chapter: frequentist approaches have several shortcomings, such as modeling language based on statistical regularities without considering word or phrase proximity, and difficulty incorporating context.

The aim of this chapter is to address the first of those points. This will serve as an introduction to *embeddings*, the language representations at the core of modern language models used in everyday tools like `DeepL` or `ChatGPT`.

1.1 Data Used

We will continue our exploration of literature using the same three English-language authors:

- Edgar Allan Poe (EAP);
- HP Lovecraft (HPL);
- Mary Wollstonecraft Shelley (MWS).

The dataset is available in a CSV file hosted on `Github`, and can be directly downloaded from: https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.csv.

To explore the topic of *embeddings*, we will use a language modeling task: predicting the author of a given text. A language model represents a text or language as a probability distribution over terms (usually words).

> **i** Sources of Inspiration
>
> This chapter is inspired by several online resources:
>
> - A first notebook on `Kaggle` and a second one;
> - A Github repository;

1.2 Required Packages

As in the previous section, we need to install specialized NLP libraries along with their dependencies. This tutorial will use several libraries, including some that depend on `PyTorch`, which is a large framework.

> **!** PyTorch on SSPCloud
>
> **The following note is only relevant for users of `SSPCloud`.**
>
> The standard `Python` services on `SSPCloud` (such as `vscode-python` and `jupyter-python`) do not include `PyTorch` by default. This library is quite large (around 600MB) and requires specific configuration to work seamlessly across different software environments. For ecological sustainability, this enhanced environment is not provided by default. However, when needed, an environment with `PyTorch` preinstalled is available.
>
> To access it, simply start a `vscode-pytorch` or `jupyter-pytorch` service. If you used one of the buttons above, this pre-configured service was automatically launched for you.

```
!pip install numpy pandas spacy transformers scikit-learn langchain_community
```

Next, since we will be using the `SpaCy` library with a corpus of English texts, we need to download the English NLP model. For this, you can refer to the official `SpaCy` documentation, which is extremely well-designed.

```
!python -m spacy download en_core_web_sm
```

2 Data Preparation

We will once again use the `spooky` dataset:

```
import pandas as pd

data_url  =  'https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.
csv'
spooky_df = pd.read_csv(data_url)
```

The dataset pairs each author with a sentence they wrote:

```
spooky_df.head()
```

	id	text	author
0	id26305	This process, however, afforded me no means of...	EAP
1	id17569	It never once occurred to me that the fumbling...	HPL
2	id11008	In his left hand was a gold snuff box, from wh...	EAP
3	id27763	How lovely is spring As we looked from Windsor...	MWS
4	id12958	Finding nothing else, not even gold, the Super...	HPL

2.1 Preprocessing

As discussed in the previous chapter, the first step in any work with textual data is often *preprocessing*, which typically includes tokenization and text cleaning.

Here, we will stick to minimal preprocessing: removing punctuation and stop words (for visualization and count-based vectorization methods).

To begin the cleaning process, we will use the `en_core_web_sm` model from `Spacy`

```
import spacy
nlp = spacy.load('en_core_web_sm')
```

We will use a `spacy pipe` that automates and parallelizes a number of operations. *Pipes* in NLP are similar to `scikit` pipelines or `pandas` pipes. They are well-suited tools for industrializing various *preprocessing* tasks:

```
from typing import List
import spacy

def clean_docs(
    texts: List[str],
    remove_stopwords: bool = False,
    n_process: int = 4,
    remove_punctuation: bool = True
) -> List[str]:
```

```
    """
    Cleans a list of text documents by tokenizing, optionally removing stopwords, and optionally
removing punctuation.

    Parameters:
        texts (List[str]): List of text documents to clean.
        remove_stopwords (bool): Whether to remove stopwords. Default is False.
        n_process (int): Number of processes to use for processing. Default is 4.
        remove_punctuation (bool): Whether to remove punctuation. Default is True.

    Returns:
        List[str]: List of cleaned text documents.
    """
    # Load spacy's nlp model
    docs = nlp.pipe(
        texts,
        n_process=n_process,
        disable=['parser', 'ner', 'lemmatizer', 'textcat']
    )

    # Pre-load stopwords for faster checking
    stopwords = set(nlp.Defaults.stop_words)

    # Process documents
    docs_cleaned = (
        ' '.join(
            tok.text.lower().strip()
            for tok in doc
            if (not remove_punctuation or not tok.is_punct) and
               (not remove_stopwords or tok.text.lower() not in stopwords)
        )
        for doc in docs
    )

    return list(docs_cleaned)
```

We apply the `clean_docs` function to our `pandas` column. Since `pandas.Series` are iterable, they behave like lists and work very well with our `spacy` pipe.

```
spooky_df['text_clean'] = clean_docs(spooky_df['text'])
```

```
spooky_df.head()
```

	id	text	author	text_clean
0	id26305	This process, however, afforded me no means of...	EAP	this process however afforded me no means of a...
1	id17569	It never once occurred to me that the fumbling...	HPL	it never once occurred to me that the fumbling...
2	id11008	In his left hand was a gold snuff box, from wh...	EAP	in his left hand was a gold snuff box from whi...
3	id27763	How lovely is spring As we looked from Windsor...	MWS	how lovely is spring as we looked from windsor...
4	id12958	Finding nothing else, not even gold, the Super...	HPL	finding nothing else not even gold the superin...

2.2 Encoding the Target Variable

We perform a simple encoding of the target variable: there are three categories (authors), represented by integers 0, 1, and 2. For this, we use `Scikit`'s `LabelEncoder`, previously introduced in the modeling section. We will use the `fit_transform` method, which conveniently combines fitting (i.e., creating a mapping between numerical values and labels) and transforming the same column in one step.

We can check the classes of our `LabelEncoder`:

```
array(['EAP', 'HPL', 'MWS'], dtype=object)
```

2.3 Creating the Training and Test Sets

We set aside a test sample (20%) before performing any analysis (even descriptive). This ensures a rigorous evaluation of our models at the end, since these data will never have been seen during training.

Our initial dataset is not balanced—some authors have more texts than others. To ensure fair evaluation of our model, we will stratify the sampling so that the training and test sets contain a similar distribution of authors.

```
from sklearn.model_selection import train_test_split

y = spooky_df["author"]
X = spooky_df['text_clean']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Preview of the first element in `X_train`:

```
'this process however afforded me no means of ascertaining the dimensions of my dungeon as i
might make its circuit and return to the point whence i set out without being aware of the fact
so perfectly uniform seemed the wall'
```

3 Vectorization Using the *Bag of Words* Approach

Representing our texts as a bag of words allows us to vectorize the corpus and thus obtain a numerical representation of each text. From there, we can perform various types of modeling tasks.

Let's define our vector representation using TF-IDF with `Scikit`:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline

pipeline_tfidf = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=10000)),
])
pipeline_tfidf
```

	steps steps: list of tuples **List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define `fit`. All non-last steps must also define `transform`. See :ref:`Combining Estimators <combining_estimators>` for more details.**	**[('tfidf', ...)]**
	transform_input transform_input: list of str, default=None The names of the :term:`metadata` parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to ``fit`` (other than ``X``) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:`metadata routing <metadata_routing>`. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using ``sklearn.set_config(enable_metadata_routing=True)``. .. versionadded:: 1.6	None
	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the	None

	transformer instance given to the pipeline cannot be inspected directly. Use the attribute ``named_steps`` or ``steps`` to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:`sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py` for an example on how to enable caching.	
	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False
	max_features max_features: int, default=None **If not None, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus. Otherwise, all features are used.** **This parameter is ignored if vocabulary is not None.**	10000
	input input: {'filename', 'file', 'content'}, default='content' - If ``'filename'``, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze. - If ``'file'``, the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory. - If ``'content'``, the input is expected to be a sequence of items that can be of type string or byte.	'content'

	encoding encoding: str, default='utf-8' If bytes or files are given to analyze, this encoding is used to decode.	'utf-8'
	decode_error decode_error: {'strict', 'ignore', 'replace'}, default='strict' Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given `encoding`. By default, it is 'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.	'strict'
	strip_accents strip_accents: {'ascii', 'unicode'} or callable, default=None Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed. Both 'ascii' and 'unicode' use NFKD normalization from :func:`unicodedata.normalize`.	None
	lowercase lowercase: bool, default=True Convert all characters to lowercase before tokenizing.	True
	preprocessor preprocessor: callable, default=None Override the preprocessing (string transformation) stage while preserving the tokenizing and n-grams generation steps. Only applies if ``analyzer`` is not callable.	None

	tokenizer tokenizer: callable, default=None Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if ``analyzer == 'word'``.	None
	analyzer analyzer: {'word', 'char', 'char_wb'} or callable, default='word' Whether the feature should be made of word or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space. If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input. .. versionchanged:: 0.21 Since v0.21, if ``input`` is ``'filename'`` or ``'file'``, the data is first read from the file and then passed to the given callable analyzer.	'word'
	stop_words stop_words: {'english'}, list, default=None If a string, it is passed to _check_stop_list and the appropriate stop list is returned. 'english' is currently the only supported string value. There are several known issues with 'english' and you should consider an alternative (see :ref:`stop_words`). If a list, that list is assumed to contain stop words, all of which will be removed from the resulting tokens. Only applies if ``analyzer == 'word'``.	None

	If None, no stop words will be used. In this case, setting `max_df` to a higher value, such as in the range (0.7, 1.0), can automatically detect and filter stop words based on intra corpus document frequency of terms.	
	token_pattern token_pattern: str, default=r"(?u)\\b\\w\\w+\\b" Regular expression denoting what constitutes a "token", only used if ``analyzer == 'word'``. The default regexp selects tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator). If there is a capturing group in token_pattern then the captured group content, not the entire match, becomes the token. At most one capturing group is permitted.	'(?u)\\b\\w\\w+\\b'
	ngram_range ngram_range: tuple (min_n, max_n), default=(1, 1) The lower and upper boundary of the range of n-values for different n-grams to be extracted. All values of n such that min_n <= n <= max_n will be used. For example an ``ngram_range`` of ``(1, 1)`` means only unigrams, ``(1, 2)`` means unigrams and bigrams, and ``(2, 2)`` means only bigrams. Only applies if ``analyzer`` is not callable.	(1, ...)
	max_df max_df: float or int, default=1.0 When building the vocabulary ignore terms that have a docu-	1.0

	ment frequency strictly higher than the given threshold (corpus-specific stop words). If float in range [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	
	min_df min_df: float or int, default=1 When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float in range of [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1
	vocabulary vocabulary: Mapping or iterable, default=None Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or an iterable over terms. If not given, a vocabulary is determined from the input documents.	None
	binary binary: bool, default=False If True, all non-zero term counts are set to 1. This does not mean outputs will have only 0/1 values, only that the tf term in tf-idf is binary. (Set `binary` to True, `use_idf` to False and `norm` to None to get 0/1 outputs).	False
	dtype dtype: dtype, default=float64	<class 'numpy.float64'>

	Type of the matrix returned by fit_transform() or transform().	
	norm norm: {'l1', 'l2'} or None, default='l2' Each output row will have unit norm, either: - 'l2': Sum of squares of vector elements is 1. The cosine similarity between two vectors is their dot product when l2 norm has been applied. - 'l1': Sum of absolute values of vector elements is 1. See :func:`~sklearn.preprocessing.normalize`. - None: No normalization.	'l2'
	use_idf use_idf: bool, default=True Enable inverse-document-frequency reweighting. If False, idf(t) = 1.	True
	smooth_idf smooth_idf: bool, default=True Smooth idf weights by adding one to document frequencies, as if an extra document was seen containing every term in the collection exactly once. Prevents zero divisions.	True
	sublinear_tf sublinear_tf: bool, default=False Apply sublinear tf scaling, i.e. replace tf with 1 + log(tf).	False

Let's go ahead and train our model to vectorize the text using the TF-IDF method. At this stage, we are not yet concerned with evaluation, so we will train on the entire dataset, not just `X_train`.

```
pipeline_tfidf.fit(spooky_df['text_clean'])
```

	steps steps: list of tuples **List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define `fit`. All non-last steps must also define `trans-**	**[('tfidf', ...)]**

	form`. See :ref:`Combining Estimators <combining_estimators>` for more details.	
	transform_input transform_input: list of str, default=None The names of the :term:`metadata` parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to ``fit`` (other than ``X``) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:`metadata routing <metadata_routing>`. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using ``sklearn.set_config(enable_metadata_routing=True)``. .. versionadded:: 1.6	None
	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute ``named_steps`` or ``steps`` to inspect estimators within the pipeline. Caching the transformers is advantageous	None

	when fitting is time consuming. See :ref:`sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py` for an example on how to enable caching.	
	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False
	max_features max_features: int, default=None **If not None, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus. Otherwise, all features are used.** **This parameter is ignored if vocabulary is not None.**	10000
	input input: {'filename', 'file', 'content'}, default='content' - If `'filename'`, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze. - If `'file'`, the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory. - If `'content'`, the input is expected to be a sequence of items that can be of type string or byte.	'content'
	encoding encoding: str, default='utf-8' If bytes or files are given to analyze, this encoding is used to decode.	'utf-8'
	decode_error decode_error: {'strict', 'ignore', 'replace'}, default='strict'	'strict'

	Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given `encoding`. By default, it is 'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.	
	strip_accents strip_accents: {'ascii', 'unicode'} or callable, default=None Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed. Both 'ascii' and 'unicode' use NFKD normalization from :func:`unicodedata.normalize`.	None
	lowercase lowercase: bool, default=True Convert all characters to lowercase before tokenizing.	True
	preprocessor preprocessor: callable, default=None Override the preprocessing (string transformation) stage while preserving the tokenizing and n-grams generation steps. Only applies if ``analyzer`` is not callable.	None
	tokenizer tokenizer: callable, default=None Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if ``analyzer == 'word'``.	None
	analyzer analyzer: {'word', 'char', 'char_wb'} or callable, de-	'word'

	fault='word' Whether the feature should be made of word or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space. If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input. .. versionchanged:: 0.21 Since v0.21, if ``input`` is ``'filename'`` or ``'file'``, the data is first read from the file and then passed to the given callable analyzer.	
	stop_words stop_words: {'english'}, list, default=None If a string, it is passed to _check_stop_list and the appropriate stop list is returned. 'english' is currently the only supported string value. There are several known issues with 'english' and you should consider an alternative (see :ref:`stop_words`). If a list, that list is assumed to contain stop words, all of which will be removed from the resulting tokens. Only applies if ``analyzer == 'word'``. If None, no stop words will be used. In this case, setting `max_df` to a higher value, such as in the range (0.7, 1.0), can automatically detect and filter stop words based on intra corpus document frequency of terms.	None
	token_pattern token_pattern: str, default=r"(?u)\\b\\w\\w+\\b"	'(?u)\\b\\w\\w+\\b'

	Regular expression denoting what constitutes a "token", only used if ``analyzer == 'word'``. The default regexp selects tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator). If there is a capturing group in token_pattern then the captured group content, not the entire match, becomes the token. At most one capturing group is permitted.	
	ngram_range ngram_range: tuple (min_n, max_n), default=(1, 1) The lower and upper boundary of the range of n-values for different n-grams to be extracted. All values of n such that min_n <= n <= max_n will be used. For example an ``ngram_range`` of ``(1, 1)`` means only unigrams, ``(1, 2)`` means unigrams and bigrams, and ``(2, 2)`` means only bigrams. Only applies if ``analyzer`` is not callable.	(1, ...)
	max_df max_df: float or int, default=1.0 When building the vocabulary ignore terms that have a document frequency strictly higher than the given threshold (corpus-specific stop words). If float in range [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts.	1.0

	This parameter is ignored if vocabulary is not None.	
	min_df min_df: float or int, default=1 When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float in range of [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1
	vocabulary vocabulary: Mapping or iterable, default=None Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or an iterable over terms. If not given, a vocabulary is determined from the input documents.	None
	binary binary: bool, default=False If True, all non-zero term counts are set to 1. This does not mean outputs will have only 0/1 values, only that the tf term in tf-idf is binary. (Set `binary` to True, `use_idf` to False and `norm` to None to get 0/1 outputs).	False
	dtype dtype: dtype, default=float64 Type of the matrix returned by fit_transform() or transform().	<class 'numpy.float64'>
	norm norm: {'l1', 'l2'} or None, default='l2' Each output row will have unit norm, either: - 'l2': Sum of squares of vector elements is 1. The cosine similarity between two vectors	'l2'

	is their dot product when l2 norm has been applied. - 'l1': Sum of absolute values of vector elements is 1. See :func:`~sklearn.preprocessing.normalize`. - None: No normalization.	
	use_idf use_idf: bool, default=True Enable inverse-document-frequency reweighting. If False, idf(t) = 1.	True
	smooth_idf smooth_idf: bool, default=True Smooth idf weights by adding one to document frequencies, as if an extra document was seen containing every term in the collection exactly once. Prevents zero divisions.	True
	sublinear_tf sublinear_tf: bool, default=False Apply sublinear tf scaling, i.e. replace tf with 1 + log(tf).	False

Name	Type	Value
fixed_vocabulary_ fixed_vocabulary_: bool True if a fixed vocabulary of term to indices mapping is provided by the user.	bool	False
idf_ idf_: array of shape (n_features,) The inverse document frequency (IDF) vector; only defined if ``use_idf`` is True.	ndarray[float64](10000,)	[8.4 ,7.48,9.09,...,8.8 ,9.09,9.27]
vocabulary_ vocabulary_: dict A mapping of terms to feature indices.	dict	{'abandon': np.int64(0), 'ab...ed': np.int64(1), 'ab...nt': np.int64(2), 'abaout': np.int64(3), ...}

abandon
abandoned
abandonment
abaout

abbey
abdication
abdul
abernethy
abhor
abhorred
abhorrence
abhorrent
abilities
ability
abject
able
abnormal
abnormality
abnormally
abode
abodes
about
above
abroad
abrupt
abruptly
absence
absent
absolute
absolutely
absorbed
absorbing
abstract
abstraction
absurd
absurdity
absurdly
abundance
abundant
abundantly
abysmal
abyss
abysses
academy
accelerated
accent
accents

accept
accepted
access
accident
accidental
accidentally
accidents
accommodation
accompanied
accompany
accompanying
accomplish
accomplished
accomplishing
accomplishment
accomplishments
accord
accordance
accorded
according
accordingly
accost
accosted
account
accounted
accounting
accounts
accumulated
accuracy
accurate
accurately
accursed
accuse
accused
accustomed
acetylene
achieve
achieved
achievement
achievements
aching
acknowledge
acknowledged

acquaintance
acquaintances
acquainted
acquire
acquired
acquirement
acquiring
acre
across
act

3.1 Finding the Most Similar Text

First, we can look for the text that is closest—according to TF-IDF similarity—to a given sentence. Let's take the following example:

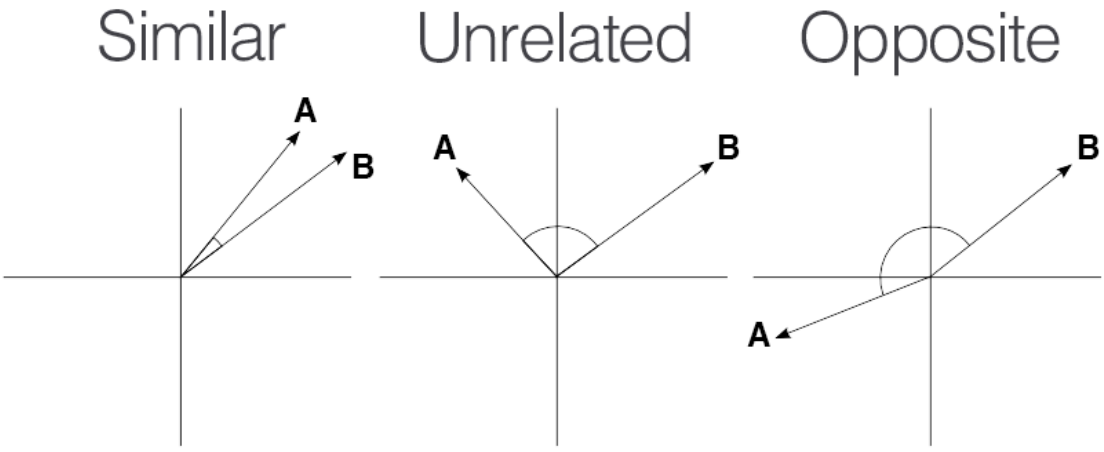
```
text = "He was afraid by Frankenstein monster"
```

How can we find the text most similar to this one? We need to transform our sentence into the same vector representation, then compare it to the other texts using that same form.

This is essentially an information retrieval task—a classic NLP use case—implemented, for example, by search engines. Since the term "Frankenstein" is quite distinctive, we should be able to identify similarities with other texts written by Mary Shelley using TF-IDF.

A metric commonly used to compare vectors is cosine similarity. This is a central measure in modern NLP. While it is more meaningful with dense vectors (which we'll explore soon), it still provides a useful exercise for understanding similarity between two vectors, even when those vectors are sparse, as in the bag-of-words approach.

If each dimension of a vector represents a direction, cosine similarity measures the angle between two vectors. The smaller the angle, the closer the vectors.



3.1.a With `Scikit-Learn`

Exercise 1: Similarity Search with TF-IDF

1. Use the `transform` method to vectorize the entire training corpus.

2. Assuming your vectorized training set is named `X_train_tfidf`, you can convert it to a *DataFrame* with the following command:

```
X_train_tfidf = pd.DataFrame(
    X_train_tfidf.todense(), columns=pipeline_tfidf.get_feature_names_out()
)
```

3. Use `Scikit`'s `cosine_similarity` method to compute cosine similarity between your vectorized text and the training corpus using this code:

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

cosine_similarities = cosine_similarity(
    X_train_tfidf,
    pipeline_tfidf.transform([text])
).flatten()

top_4_indices = np.argsort(cosine_similarities)[-4:][::-1]  # Descending sort
top_4_similarities = cosine_similarities[top_4_indices]
```

4. Retrieve the corresponding documents. Are you satisfied with the result? Do you understand what happened?

At the end of the exercise, the 4 most similar texts are:

	text	author	score
8181	Listen to me, Frankenstein.	MWS	0.402964
8606	He was gazing at me gaspingly and fascinatedly...	HPL	0.330177
14550	The light is dimmer and the gods are afraid. . .	HPL	0.314670
11366	I screamed aloud that I was not afraid; that I...	HPL	0.311235

3.1.b With `Langchain`

This approach to computing text similarity is rather tedious with `Scikit`. With the rapid development of `Python` applications leveraging language models, a rich ecosystem has emerged to make these tasks achievable in just a few lines of code.

Among the most valuable tools is `Langchain`, a high-level `Python` ecosystem for building production-ready pipelines using textual data.

We will proceed here in two steps:

- Create a *retriever*, which involves vectorizing our corpus (texts from the three authors) using TF-IDF and storing it in a vector database.
- Vectorize our search query (`text`, created earlier) on the fly and retrieve its closest match from the vector database.

Vectorizing our corpus is very straightforward using `Langchain`, as `Scikit`'s `TfidfVectorizer` is wrapped in a dedicated module provided by `Langchain`.

```
from langchain_community.retrievers import TFIDFRetriever
from langchain_community.document_loaders import DataFrameLoader

loader = DataFrameLoader(spooky_df, page_content_column="text_clean")

retriever = TFIDFRetriever.from_documents(
```

```
    loader.load()
)
```

```
/tmp/ipykernel_339/3052793950.py:1: DeprecationWarning: `langchain-community` is being sunset
and is no longer actively maintained. See https://github.com/langchain-ai/langchain-community/
issues/674 for details and migration guidance toward standalone integration packages.
  from langchain_community.retrievers import TFIDFRetriever
```

This `retriever` object serves as an entry point to our corpus. `Langchain` is particularly valuable in NLP projects because it provides standardized entry points, allowing you to easily switch out vectorizers without needing to change how the results are used at the end of the pipeline.

The `invoke` method is used to find the most similar vectors to our search query:

```
retriever.invoke(text)
```

```
[Document(metadata={'id': 'id12587', 'text': 'Listen to me, Frankenstein.', 'author': 'MWS',
'author_encoded': 2}, page_content='listen to me frankenstein'),
 Document(metadata={'id': 'id09284', 'text': 'I screamed aloud that I was not afraid; that I never
could be afraid; and others screamed with me for solace.', 'author': 'HPL', 'author_encoded':
1}, page_content='i screamed aloud that i was not afraid that i never could be afraid and others
screamed with me for solace'),
 Document(metadata={'id': 'id09797', 'text': 'It seemed to be a sort of monster, or symbol
representing a monster, of a form which only a diseased fancy could conceive.', 'author': 'HPL',
'author_encoded': 1}, page_content='it seemed to be a sort of monster or symbol representing a
monster of a form which only a diseased fancy could conceive'),
 Document(metadata={'id': 'id10816', 'text': 'And, as I have implied, it was not of the dead man
himself that I became afraid.', 'author': 'HPL', 'author_encoded': 1}, page_content='and as i
have implied it was not of the dead man himself that i became afraid')]
```

The output is a `Langchain` object, which is not convenient for our purposes here. We convert it into a *DataFrame*:

```
documents = []
for best_echoes in retriever.invoke(text):
    documents += [{**best_echoes.metadata, **{"text_clean": best_echoes.page_content}}]

documents = pd.DataFrame(documents)
```

We can add the similarity score column to this *DataFrame*:

We do indeed retrieve the same documents:

	id	text	author	author_encoded	text_clean	score
0	id12587	Listen to me, Frankenstein.	MWS	2	listen to me frankenstein	0.402964
1	id09284	I screamed aloud that I was not afraid; that I...	HPL	1	i screamed aloud that i was not afraid that i ...	0.311235
2	id09797	It seemed to be a sort of monster, or symbol r...	HPL	1	it seemed to be a sort of monster or symbol re...	0.295587
3	id10816	And, as I have implied, it was not of the dead...	HPL	1	and as i have implied it was not of the dead m...	0.261818

> **i** The BM25 Metric
>
> BM25 is a probabilistic relevance-based information retrieval model, similar to TF-IDF. It is commonly used in search engines to rank documents relative to a query.

BM25 combines term frequency (TF), inverse document frequency (IDF), and a normalization based on document length. In other words, it improves on TF-IDF by adjusting scores based on string length to avoid overemphasizing longer documents.

BM25 performs particularly well in environments where documents vary in length and content. This is why search engines such as `Elasticsearch` have made it a cornerstone of their search mechanisms.

Why aren't all results relevant? We can anticipate several reasons.

The first hypothesis is that we're training our vectorizer on a biased corpus. While "Frankenstein" is a rare term, it appears more frequently in our dataset than in general English usage. The inverse document frequency is thus biased against the term: its appearance should be a much stronger indicator that the text belongs to Mary Shelley. While addressing this might slightly improve relevance, it's not the core issue.

The frequentist approach assumes all terms are equally distinct. A sentence containing the word *"creature"* won't get a higher score when searching for *"monster"*. Again, we've treated our corpus as a bag where words are independent—there's no increased likelihood of encountering *"Frankenstein"* after *"doctor"*. These limitations point us toward the topic of *embeddings*. Even though the frequentist method may seem a bit *old school*, it's not useless and often provides a "tough to beat baseline." In fields like information extraction from short texts, where every term carries strong signal, this approach is often effective.

3.2 Finding the Closest Author: An Introduction to the Naive Bayes Classifier

Before diving into *embeddings*, let's explore a slightly different use case within our probabilistic framework. Suppose we want to predict the author of a given text. If our previous intuition holds—certain words are more likely to appear in texts by specific authors—then we can train an automatic classification algorithm to predict the author based on the text.

The most natural method for this task is the Naive Bayes classifier. This model is a perfect fit for the frequentist approach we've used so far, as it relies on the probabilities of word occurrences per author.

The Naive Bayes classifier applies a decision rule: it selects the most probable class given the observed structure of the document—i.e., the words that appear in it.

In other words, we choose the class $\hat{c}$ that is most probable given the terms in document d.

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(c|d) = \arg\max_{c \in \mathcal{C}} \frac{\mathbb{P}(d|c)\mathbb{P}(c)}{\mathbb{P}(d)} \tag{1}$$

As is common in Bayesian estimation, we can ignore constant terms such as $\mathbb{P}(d)$. The definition of the predicted class can thus be reformulated as follows:

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(d|c)\mathbb{P}(c) \tag{2}$$

The bag-of-words assumption comes into play here. A document d is considered a collection of words w_i, where word order is irrelevant. In other words, we can build a model based on individual words without involving conditional probabilities related to their order. The second strong assumption is the naive assumption from which the method gets its name: the probability of drawing a word depends only on the category c to which the document belongs. In other words, a document is treated as a sequence of independent word draws, where the probability depends solely on the author.

As explained in the dedicated box, under these assumptions, the classifier can be rewritten in the following form

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(c) \prod_{w \in \mathcal{W}} \mathbb{P}(w|c)$$

where $\mathcal{W}$ is the set of words in the corpus (our vocabulary).

Empirically, this is a supervised learning task where the *label* is the document class and the *features* are our vectorized words. In practice, the probabilities are estimated from word counts in the corpus and the distribution of document types.

While it is possible to compute all these quantities manually, `Scikit` makes it easy to implement a Naive Bayes estimator after vectorizing the corpus, as shown in the next exercise. However, this may introduce a practical issue: ideally, the test set should not contain new words that were not in the training set, since these new dimensions did not exist during training. In practice, the most common solution is the one adopted here: these words are ignored.

Exercise 2: The Naive Bayes Classifier

1. Starting from the previous example, define a *pipeline* that vectorizes each document (using `CountVectorizer` instead of `TFIDFVectorizer`) and performs prediction using a Naive Bayes model.
2. Train this model and make predictions on the test set.
3. Evaluate the performance of your model.
4. Make a prediction for the sentence we previously stored in the `text` variable. Do you get the expected result?
5. Examine the predicted probabilities (using the `predict_proba` method).

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

pipeline = Pipeline([
    ('vectorizer', CountVectorizer()),
    ('classifier', MultinomialNB())
])

# Train the pipeline on the training data
pipeline.fit(X_train, y_train)
```

	steps steps: list of tuples List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define `fit`. All non-last steps must also define `transform`. See :ref:`Combining Estimators <combining_estimators>` for more details.	[('vectorizer', ...), ('classifier', ...)]
	transform_input transform_input: list of str, default=None The names of the :term:`metadata` parameters that should be	None

	transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to ``fit`` (other than ``X``) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:`metadata routing <metadata_routing>`. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using ``sklearn.set_config(enable_metadata_routing=True)``. .. versionadded:: 1.6	
	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute ``named_steps`` or ``steps`` to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:`sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py` for an example on how to enable caching.	None
	verbose verbose: bool, default=False If True, the time elapsed while	False

	fitting each step will be printed as it is completed.	

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) The classes labels. Only exist if the last step of the pipeline is a classifier.	ndarray[<U3](3,)	['EAP','HPL','MWS']

	input input: {'filename', 'file', 'content'}, default='content' **- If `'filename'`, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze.** **- If `'file'`, the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory.** **- If `'content'`, the input is expected to be a sequence of items that can be of type string or byte.**	**'content'**
	encoding encoding: str, default='utf-8' If bytes or files are given to analyze, this encoding is used to decode.	'utf-8'
	decode_error decode_error: {'strict', 'ignore', 'replace'}, default='strict' Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given `encoding`. By default, it is 'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.	'strict'
	strip_accents strip_accents: {'ascii', 'unicode'} or callable, default=None	None

	Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed. Both 'ascii' and 'unicode' use NFKD normalization from :func:`unicodedata.normalize`.	
	lowercase lowercase: bool, default=True Convert all characters to lowercase before tokenizing.	True
	preprocessor preprocessor: callable, default=None Override the preprocessing (strip_accents and lowercase) stage while preserving the tokenizing and n-grams generation steps. Only applies if ``analyzer`` is not callable.	None
	tokenizer tokenizer: callable, default=None Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if ``analyzer == 'word'``.	None
	stop_words stop_words: {'english'}, list, default=None If 'english', a built-in stop word list for English is used. There are several known issues with 'english' and you should consider an alternative (see :ref:`stop_words`). If a list, that list is assumed to contain stop words, all of which will be removed from the resulting tokens. Only applies if ``analyzer ==	None

	'word'``. If None, no stop words will be used. In this case, setting `max_df` to a higher value, such as in the range (0.7, 1.0), can automatically detect and filter stop words based on intra corpus document frequency of terms.	
	token_pattern token_pattern: str or None, default=r"(?u)\\b\\w\\w+\\b" Regular expression denoting what constitutes a "token", only used if ``analyzer == 'word'``. The default regexp select tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator). If there is a capturing group in token_pattern then the captured group content, not the entire match, becomes the token. At most one capturing group is permitted.	'(?u)\\b\\w\\w+\\b'
	ngram_range ngram_range: tuple (min_n, max_n), default=(1, 1) The lower and upper boundary of the range of n-values for different word n-grams or char n-grams to be extracted. All values of n such such that min_n <= n <= max_n will be used. For example an ``ngram_range`` of ``(1, 1)`` means only unigrams, ``(1, 2)`` means unigrams and bigrams, and ``(2, 2)`` means only bigrams. Only applies if ``analyzer`` is not callable.	(1, ...)
	analyzer analyzer: {'word', 'char', 'char_wb'} or callable, default='word'	'word'

	Whether the feature should be made of word n-gram or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space. If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input. .. versionchanged:: 0.21 Since v0.21, if ``input`` is ``filename`` or ``file``, the data is first read from the file and then passed to the given callable analyzer.	
	max_df max_df: float in range [0.0, 1.0] or int, default=1.0 When building the vocabulary ignore terms that have a document frequency strictly higher than the given threshold (corpus-specific stop words). If float, the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1.0
	min_df min_df: float in range [0.0, 1.0] or int, default=1 When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float, the parameter represents a proportion of documents, integer absolute counts.	1

	This parameter is ignored if vocabulary is not None.	
	max_features max_features: int, default=None If not None, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus. Otherwise, all features are used. This parameter is ignored if vocabulary is not None.	None
	vocabulary vocabulary: Mapping or iterable, default=None Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or an iterable over terms. If not given, a vocabulary is determined from the input documents. Indices in the mapping should not be repeated and should not have any gap between 0 and the largest index.	None
	binary binary: bool, default=False If True, all non zero counts are set to 1. This is useful for discrete probabilistic models that model binary events rather than integer counts.	False
	dtype dtype: dtype, default=np.int64 Type of the matrix returned by fit_transform() or transform().	<class 'numpy.int64'>

Name	Type	Value
fixed_vocabulary_ fixed_vocabulary_: bool True if a fixed vocabulary of term to indices mapping is provided by the user.	bool	False
vocabulary_ vocabulary_: dict	dict	{'ab': 0, 'aback': 1, 'abaft': 2, 'abandon': 3, ...}

A mapping of terms to feature indices.

ab
aback
abaft
abandon
abandoned
abandoning
abandonment
abaout
abased
abasement
abashed
abashment
abate
abatement
abating
abbey
abbreviation
abbé
abdicated
abdication
abdications
abdomen
abdul
abernethy
aberrancy
aberrant
aberration
aberrations
abeyance
abhor
abhorred
abhorrence
abhorrent
abigail
abijah
abilities
ability
abject
abjure
ablaze

able
ably
abnormal
abnormalities
abnormality
abnormally
aboard
abode
abodes
abolished
abominable
abomination
abominations
aboriginal
abortion
abortions
abortive
abounds
about
above
abra
abreast
abroad
abrupt
abruptly
abruptness
absence
absences
absent
absolute
absolutely
absolved
absorb
absorbed
absorbing
absorbingly
absorption
abstain
abstemious
abstract
abstracted
abstraction
abstractions

abstractly
abstruse
abstruseness
absurd
absurdities
absurdity
absurdly
absurdness
absurdum
abundance
abundant
abundantly
abuse
abused
abuses
abutted
abutting

	alpha: float or array-like of shape (n_features,), default=1.0 **Additive (Laplace/Lidstone) smoothing parameter (set alpha=0 and force_alpha=True, for no smoothing).**	**1.0**
	force_alpha: bool, default=True If False and alpha is less than 1e-10, it will set alpha to 1e-10. If True, alpha will remain unchanged. This may cause numerical errors if alpha is too close to 0. .. versionadded:: 1.2 .. versionchanged:: 1.4 The default value of `force_alpha` changed to `True`.	True
	fit_prior: bool, default=True Whether to learn class prior probabilities or not. If false, a uniform prior will be used.	True

	class_prior class_prior: array-like of shape (n_classes,), default=None Prior probabilities of the classes. If specified, the priors are not adjusted according to the data.	None

Name	Type	Value
class_count_ class_count_: ndarray of shape (n_classes,) Number of samples encountered for each class during fitting. This value is weighted by the sample weight when provided.	ndarray[float64](3,)	[6320.,4508.,4835.]
class_log_prior_ class_log_prior_: ndarray of shape (n_classes,) Smoothed empirical log probability for each class.	ndarray[float64](3,)	[−0.91,-1.25,-1.18]
classes_ classes_: ndarray of shape (n_classes,) Class labels known to the classifier	ndarray[<U3](3,)	['EAP','HPL','MWS']
feature_count_ feature_count_: ndarray of shape (n_classes, n_features) Number of samples encountered for each (class, feature) during fitting. This value is weighted by the sample weight when provided.	ndarray[float64](3, 22902)	[[1.,2.,0.,...,1.,0.,0.], [0.,0.,0.,...,0.,2.,1.], [0.,0.,1.,...,0.,0.,0.]]
feature_log_prob_ feature_log_prob_: ndarray of shape (n_classes, n_features) Empirical log probability of features given a class, ``P(x_i|y)``.	ndarray[float64](3, 22902)	[[−11.38,-10.98,-12.08,...,-11.38,-12.08,-12.08], [−11.87,-11.87,-11.87,...,-11.87,-10.77,-11.18], [−11.92,-11.92,-11.23,...,-11.92,-11.92,-11.92]]
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	22902

We obtain a satisfactory accuracy:

```
Précision: 0.8429519918283963
```

The breakdown of performance metrics is as follows:

```
              precision    recall  f1-score   support

         EAP       0.85      0.85      0.85      1580
         HPL       0.87      0.82      0.84      1127
         MWS       0.81      0.86      0.83      1209

    accuracy                           0.84      3916
   macro avg       0.84      0.84      0.84      3916
weighted avg       0.84      0.84      0.84      3916
```

Unsurprisingly, we get Mary Shelley as the predicted author:

```
np.str_('MWS')
```

Finally, when examining the predicted probabilities (question 5), we see that the prediction is very confident:

	author	proba
0	EAP	0.001675
1	HPL	0.084844
2	MWS	0.913481

Understanding the logic of the naive Bayes classifier

Suppose we are in a classification problem with classes $(c_1, ..., c_K)$ (set denoted $\mathcal{C}$). Placing ourselves within the bag-of-words framework, we can ignore the positions of words in documents, which would greatly complicate the writing of our equations.

The equation Equation 2 can be rewritten

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(w_1, ..., w_n|c)\mathbb{P}(c)$$

In the Bayesian world, we call $\mathbb{P}(w_1, ..., w_n|c)$ the likelihood and $\mathbb{P}(c)$ the prior.

The naive Bayes assumption allows us to treat a document as a sequence of random draws whose probabilities depend only on the category. In this case, drawing a sentence is a sequence of word draws and the compound probability is therefore

$$\mathbb{P}(w_1, ..., w_n|c) = \prod_{i=1}^{n} \mathbb{P}(w_i|c)$$

For example, simplifying to two classes, if the probabilities are those from Table 1, the sentence *"afraid by Doctor Frankenstein"* will have a little less than 1% chance (0.8%) of being written if the author is Mary Shelley but will be even less likely with Lovecraft (0.006%) because while *"afraid"* is very probable with him, Frankenstein is a rare event that makes this word composition unlikely.

Table 1: Fictional example of drawing probabilities

Word (w_i)	Probability for Mary Shelley	Probability for Lovecraft
Afraid	0.1	0.6
By	0.2	0.2
Doctor	0.2	0.05
Frankenstein	0.2	0.01

By combining these different equations, we get

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(c) \prod_{w \in \mathcal{W}} \mathbb{P}(w|c)$$

The empirical counterpart of $\mathbb{P}(c)$ is quite obvious: the observed frequency of each category (the authors) in our corpus. In other words,

$$\widehat{\mathbb{P}(c)} = \frac{n_c}{n_{doc}}$$

What is the empirical counterpart of $\mathbb{P}(w_i|c)$? It is the frequency of appearance of the word in question for the author. To calculate it, we simply count the number of times it appears for the author and divide by the number of words by the author.

4 The Word2Vec model, a more synthetic representation

4.1 Towards a more synthetic representation of language

The vector representation resulting from the *bag of words* approach is not very synthetic or stable and is quite crude.

If we have a small corpus, we will have problems extrapolating since new texts are very likely to bring new words, which are new feature dimensions that were not present in the training corpus. This is conceptually a problem since *machine learning* algorithms are not intended to predict on characteristics they have not been trained on[1].

Conversely, the more text we have in a corpus, the larger our vector representation will be. For example, if your bag of words has seen the entire French vocabulary, which is 60,000 words according to the French Academy (estimates being 200,000 for the English language), this results in vectors of considerable size. However, the diversity of texts is, in practice, much lower: common use of French requires around 3,000 words and most texts, especially if they are short, do not use such a complete vocabulary. This therefore implies very sparse vectors, with many 0s.

Vectorization according to this approach is therefore inefficient; the signal is poorly compressed. Dense representations, that is, of smaller dimension but all carrying information, seem more adequate to be able to generalize our language modeling. The algorithm that made this approach famous is the `Word2Vec` model, in some ways the first common ancestor of modern LLMs. The vector

[1]This remark may seem surprising while generative AIs occupy an important place in our usage. Nevertheless, we must keep in mind that while you ask new questions to AIs, you ask them in terms they know: natural language in a language present in their training corpus, digital images that are therefore interpretable by a machine, etc. In other words, your *prompt* is not, in itself, unknown to the AI, it can interpret it even if its content is new and original.

representation of `Word2Vec` is quite synthetic: the dimension of these *embeddings* is between 100 and 300.

4.2 Semantic relationships between terms

This dense representation will represent a solution to a limitation of the *bag of words* approach that we have mentioned multiple times. Each of these dimensions will represent a latent factor, that is, an unobserved variable, in the same way as principal components produced by a PCA. These latent dimensions can be interpreted as "fundamental" dimensions of language.

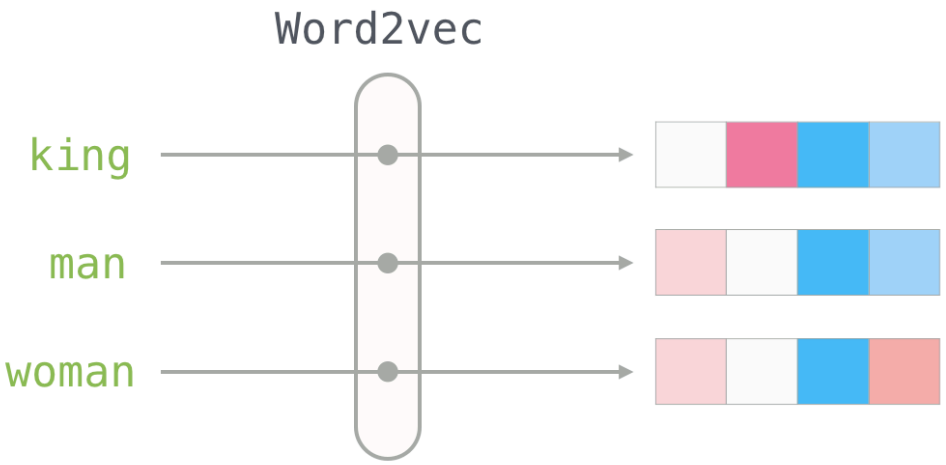


Figure 1: Illustration of the principle of Word2Vec representation (source: Jay Alammar)

For example, a human knows that a document containing the word *"King"* and another document containing the word *"Queen"* are very likely to address similar subjects. A well-trained `Word2Vec` model will capture that there exists a latent factor of type *"royalty"* and the similarity between the vectors associated with the two words will be strong.

The magic goes even further: the model will also capture that there exists a latent factor of type *"gender"*, and will allow the construction of a semantic space in which arithmetic relationships between vectors make sense. For example,

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

or, to revisit the example from the original `Word2Vec` paper [1],

$$\text{Paris} - \text{France} + \text{Italy} \approx \text{Rome}$$

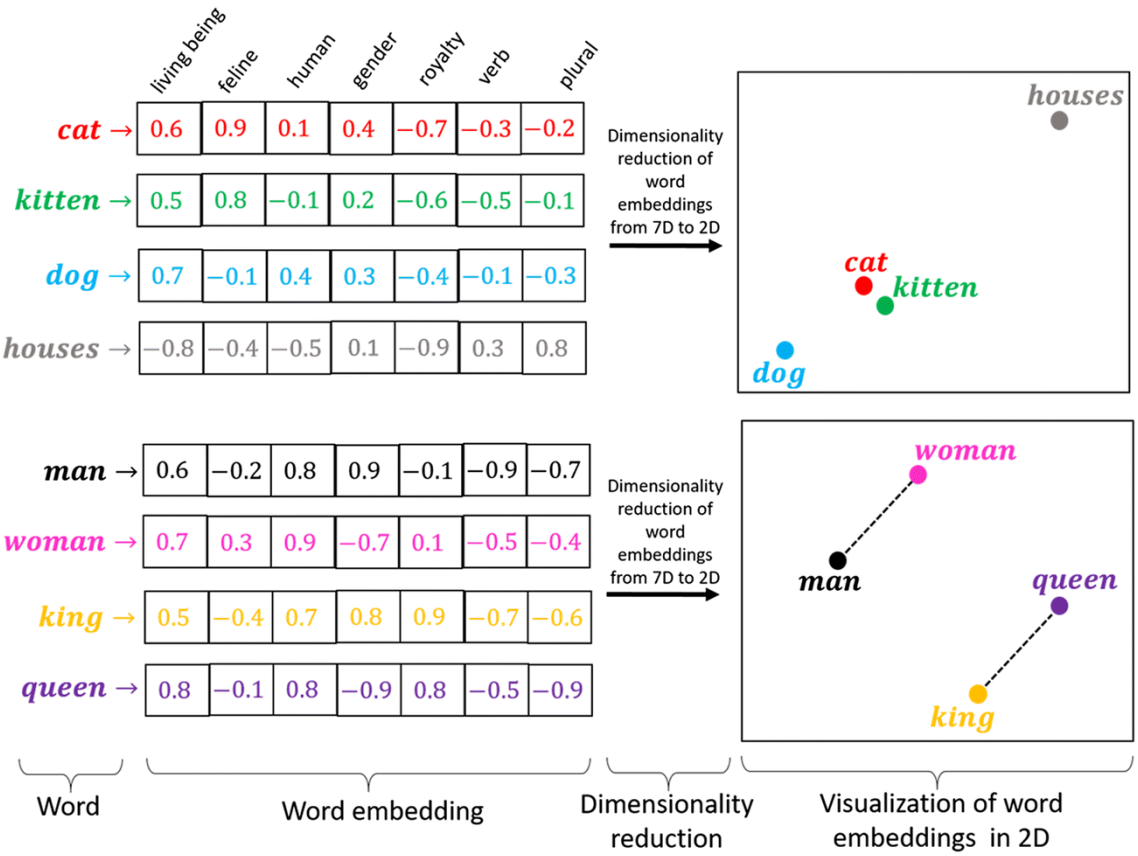


Figure 2: Illustration of lexical embedding. Source: Blog post Word Embedding: Basics

Another "miracle" of this approach is that it allows a form of transfer between languages. Since semantic relationships can be similar across languages, many common words can be mapped between languages if they share a common base (such as Western languages). This concept is the foundation of automatic translators and multilingual AI systems.

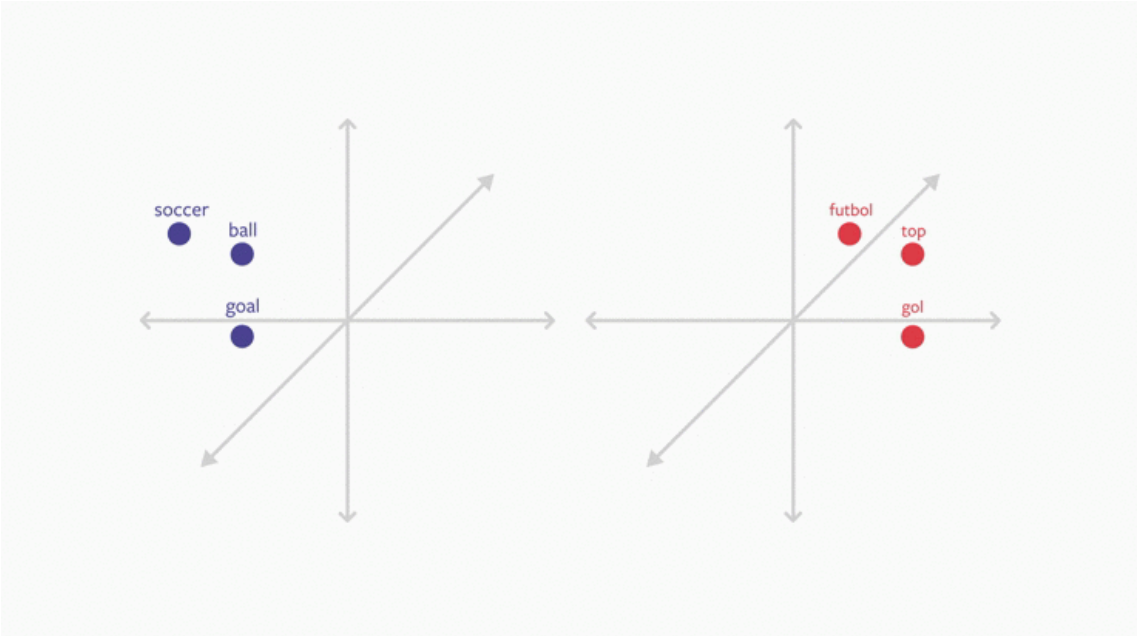


Figure 3: Example of translation between two vector representations. Source: Meta

4.3 How are these models trained?

These models are trained from a prediction task solved by a simple neural network, generally with a reinforcement approach.

The fundamental idea is that the meaning of a word is understood by looking at words that frequently appear in its neighborhood. For a given word, we will therefore try to predict the words that appear in a window around the target word.

By repeating this task many times and on a sufficiently varied corpus, we finally obtain *embeddings* for each word in the vocabulary, which present the properties discussed previously. The collection of `Wikipedia` articles is one of the preferred corpora for people who have built lexical embeddings. It indeed contains complete sentences, unlike information from social media comments, and proposes interesting connections between people, places, etc.

The context of a word is defined by a fixed-size window around this word. The window size is a parameter of the *embedding* construction. The corpus provides a large set of word-context examples, which can be used to train a neural network.

More precisely, there are two approaches, whose details we will not develop:

- *Continuous bag of words* (CBOW), where the model is trained to predict a word from its context;
- *Skip-gram*, where the model attempts to predict the context from a single word.

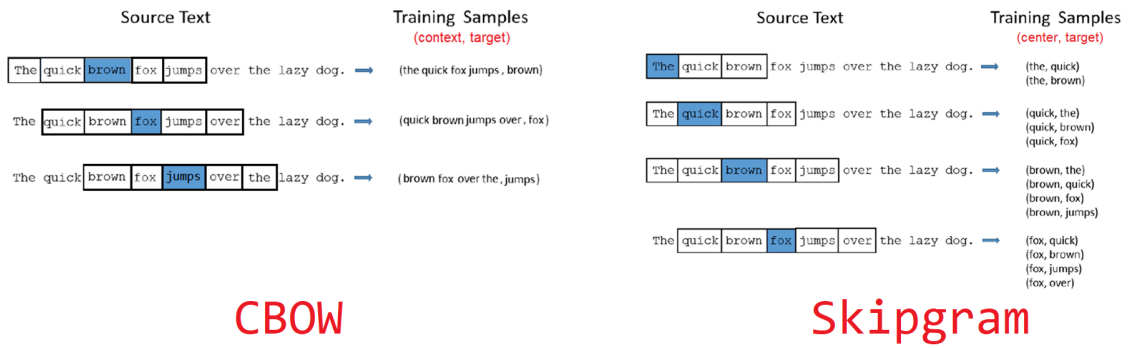


Figure 4: Illustration of the difference between CBOW and Skip-gram approaches

4.4 Related models

Several models have a direct lineage with the `Word2Vec` model although they distinguish themselves by the nature of the architecture used.

This is the case, for example, of the `GloVe` model, developed in 2014 at Stanford, which does not rely on neural networks but on the construction of a large word co-occurrence matrix. For each word, the task is to calculate the frequencies of appearance of other words in a fixed-size window around it. The obtained co-occurrence matrix is then factorized by a singular value decomposition.

The `FastText` model, developed in 2016 by a `Facebook` team, works similarly to `Word2Vec` but particularly distinguishes itself on two points:

- In addition to the words themselves, the model learns representations for character n-grams (character subsequences of size n, for example *"tar"*, *"art"* and *"rte"* are the trigrams of the word *"tarte"*), which makes it particularly robust to spelling variations;
- The model has been optimized so that its training is particularly fast.

The `FastText` model is particularly effective for automatic classification problems. INSEE uses it for example for several models of classification of textual labels in nomenclatures.

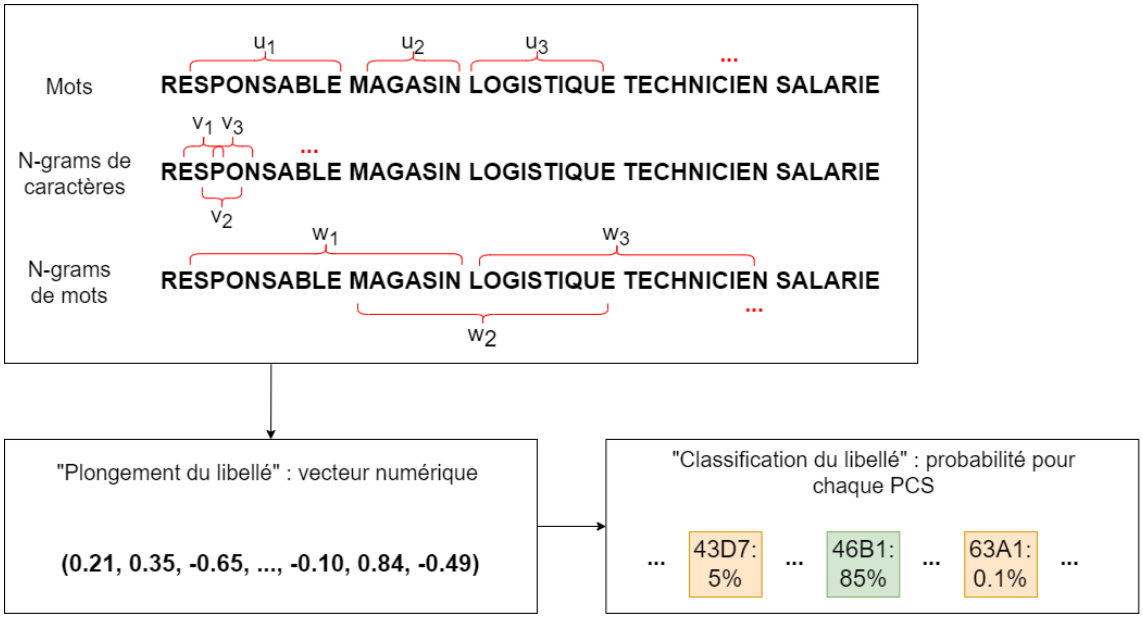


Figure 5: Illustration of the fastText model

Here is an example of an automated profession classification project in the typology of activity nomenclatures (PCS) based on a model trained by the `Fasttext` library:

```
Erreur lors de l'appel API : 503 Server Error: Service Unavailable for url: https://codification-ape-test.lab.sspcloud.fr/predict?nb_echos_max=3&prob_min=0&text_feature=data%20scientist
```

These models are inheritors of `Word2Vec` in the sense that they adopt a dense, low-dimensional vector representation of textual documents. `Word2Vec` remains a model that inherits from the bag-of-words logic. The representation of a sentence or document is a form of average of the representations of the words that compose them.

Since 2013, several revolutions have led to enriching language models to go beyond a word-by-word representation of these. Much more complex architectures to represent not only words in the form of *embeddings* but also sentences and documents are now at work and can be linked to the revolution of *transformer* architectures.

5 *Transformers*: a richer representation of language

While the `Word2Vec` model is trained contextually, its purpose is to give a vector representation of a word in an absolute manner, independent of context. For example, the term *"bank"* will have exactly the same vector representation whether it appears in the sentence *"She runs towards the sandbank"* or *"He's waiting for you on a bench in the park"*. This is a major limitation of this type of approach and we can well imagine the importance of context for language interpretation.

The objective of *transformer* architectures is to enable contextual vector representations. In other words, a word will have several vector representations, depending on its context of occurrence. These models rely on the attention mechanism [2]. Before this approach, when a model learned to vectorize a text and reached the nth word, the only memory it kept was that of the previous word. By recurrence, this meant it kept a memory of previous words but this tended to dissipate. Consequently,

for a word appearing far in the sentence, it was likely that the context from the beginning of the sentence was forgotten. In other words, in the sentence *"at the beach, he was going to explore the bank"*, it was very likely that upon reaching the word *"bank"*, the model had forgotten the beginning of the sentence which was nevertheless important for interpretation.

The objective of the attention mechanism is to create an internal memory in the model allowing, for any word in a text, to keep track of other words. Of course, not all are relevant for interpreting the text but this avoids forgetting those that are important. The main innovation of recent years in NLP has been to manage to create large-scale attention mechanisms without making the models intractable. The context windows of the most performant models are becoming immense. For example, the Llama 3.1 model (made public by Meta in July 2024) offers a context window of 128,000 *tokens*, or about 96,000 words, the equivalent of Tolkien's *Hobbit*. In other words, to deduce the subtlety of a word's meaning, this model can browse through a context as long as a novel of about 300 pages.

The two models that marked their era in the field are the `BERT` models developed in 2018 by *Google* (which was already behind `Word2Vec`) and the first version of the well-known `GPT` from `OpenAI`, which, in 2017, was the first pre-trained model based on the *transformer* architecture. These two *transformer* families differ in how they integrate context to make a prediction. `GPT` is an autoregressive model, therefore only considers the *tokens* before the one we want to predict. `BERT` uses the *tokens* to the left and right to infer context. These two major trained language models are trained by self-reinforcement, mainly on next *token* prediction tasks [3]. Since the success of `ChatGPT`, the new GPT models (from version 3 onwards) are no longer *open source*. To use them, one must therefore go through OpenAI's APIs. There are nevertheless many alternatives whose weights are open, if not *open source*[2], which allow using these LLMs through `Python`, notably through the `transformers` library developed by *Hugging Face*.

When working with small-sized corpora, it's generally a bad idea to train your own model *from scratch*. Fortunately, models pre-trained on very large corpora are available. They allow for *transfer learning*, that is, to benefit from the performance of a model that has been trained on another task or on another corpus.

Exercise 3

1. Repeat a train/test split with 500 random lines
2. Import the `all-MiniLM-L6-v2` model with the `sentence transformers` package. Encode `X_train` and `X_test`.
3. Perform a classification using a simple method, such as CVS, based on the *embeddings* produced in the previous question. As the training set is small, you can perform cross-validation.
4. Understand why the performance is worse than that of Bayes' naive classifier.

Answer to question 1:

```
random_rows = spooky_df.sample(500)
y = random_rows["author"]
X = random_rows['text']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Réponse à la question 2:

[2]Some organizations, like Meta for Llama, make available the post-training weights of their model on the *Hugging Face* platform, allowing reuse of these models if the license permits. Nevertheless, these are not *open source* models since the code used to train the models and constitute the learning corpora, derived from massive data collection by *webscraping*, and any additional annotations to make specialized versions, are not shared.

```
from sentence_transformers import SentenceTransformer
from sklearn.svm import LinearSVC

model = SentenceTransformer(
    "all-MiniLM-L6-v2", model_kwargs={"torch_dtype": "float16"}
)

X_train_vectors = model.encode(X_train.values)
X_test_vectors = model.encode(X_test.values)
```

Answer to question 3:

```
from sklearn.model_selection import cross_val_score

clf = LinearSVC(max_iter=10000, C=0.1, dual="auto")

scores = cross_val_score(
    clf, X_train_vectors, y_train,
    cv=4, scoring='f1_micro', n_jobs=4
)

print(f"CV scores {scores}")
print(f"Mean F1 {np.mean(scores)}")
```

But why, with a very complicated method, can't we beat a very simple one?

There are several possible reasons:

- the TF-IDF is a simple model, but it still performs very well (this is known as a 'tough-to-beat baseline').
- the classification of authors is a very specific and arduous task, which does not do justice to the *embeddings*. As we said earlier, the latter are particularly relevant when it comes to semantic similarity between texts (*clustering*, etc.).

In the case of our classification task, it is likely that certain words (character names, place names) are sufficient to classify in a relevant way, This is not captured by *embeddings*, which give all words the same importance.

Informations additionnelles

ℹ Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
```

```
    "cartiflette>=0.2",
    "contextily==1.6.2",
    "duckdb>=0.10.1",
    "folium>=0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables>=0.12.0",
    "gt-extras>=0.0.8",
    "ipykernel>=6.29.5",
    "jupyter>=1.1.1",
    "jupyter-cache>=1.0.0",
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true
```

```
[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.

Bibliography

[1] T. Mikolov, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, vol. 3781, 2013.

[2] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[3] "The Hugging Face Course, 2022." 2022.